

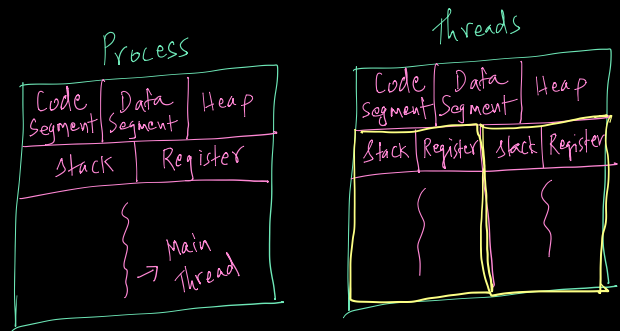
Concurrency in Python

Program: set of instructions to a computer (passive artifact on disk)

Process: instance of a program being executed (Code Segment + Data Segment + Heap + Stack + Registers) ^{per thread}

Thread: unit of execution within a process (lightweight version of process)
Threads in a process share code, heap etc. → (call stack + register + program counter)

Feature	Process (Heavyweight)	Thread (Lightweight)
Memory	Separate memory space for each process	Shared memory space within the same process
Creation Time	Takes longer (high overhead)	Faster to create (low overhead)
Context Switching	Slower (more work for OS)	Faster (less work for OS)
Communication	Uses IPC, slower	Easier communication through shared memory
Crash Impact	One process crashing doesn't affect others	A thread crash may affect other threads



Cores & Scheduler

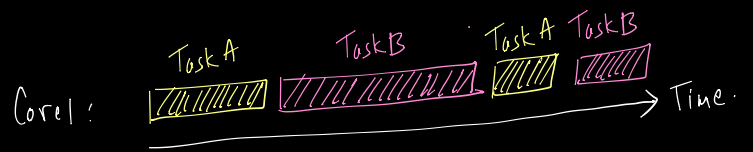
Independent processing unit inside a CPU.
Core executes one thread at a time → multiple cores → multiple process
↓
multiple threads
Scheduler is part of OS that decides which thread runs next on each core.

Concurrency: Doing many things "seemingly" at the same time.
(eg. 1 CPU core → OS is rapidly changing which program is running)
Structure of code having tasks with overlapping lifetime → 1 core → task interleaving
multicore → overlap

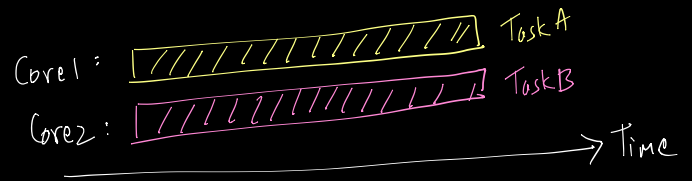
Parallelism: "Actually" doing many things at the same time.
(eg. multiple cores can execute multiple programs at the same time)
achieved by multiple cores/machines

Concurrency is about dealing with many things at once
Parallelism is about doing many things at once
— Rob Pike —

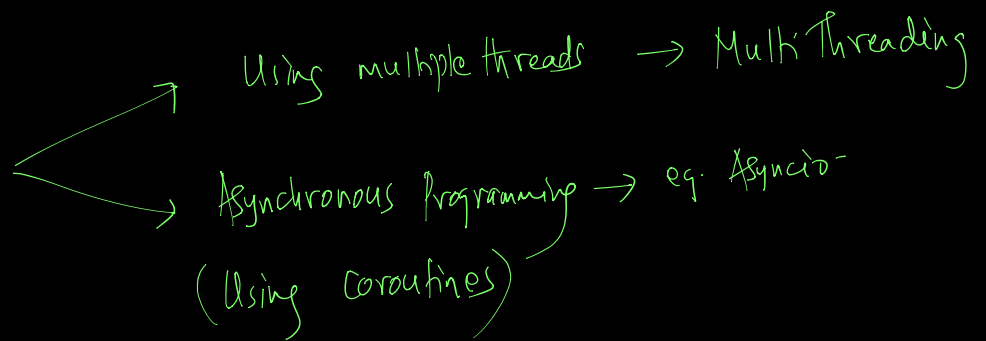
Concurrency without Parallelism.



Concurrency with Parallelism



Concurrency



Multi threading :

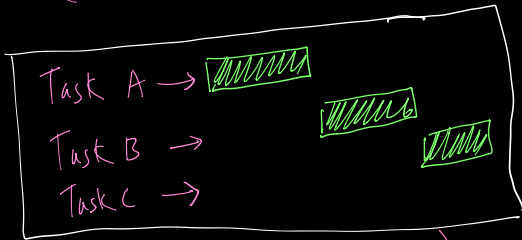
Concurrency technique in which multiple threads in a process can make progress concurrently

Asynchronous / Synchronous programming = (Control Flow of calling code)

Synchronous

Current execution "waits" for operation to complete before continuing

(Task A → Task B → Task C)

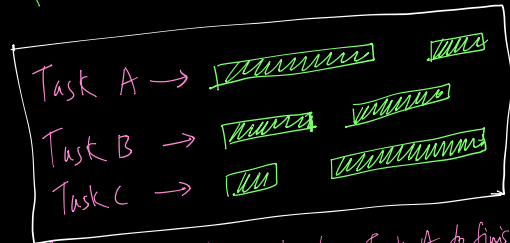


(Task B wait for Task A to finish)

→ call returns when work is finished
↓
next line runs

Asynchronous

Current execution "does not" wait & other work can make progress



(Task B doesn't wait for Task A to finish)

→ task can suspend itself and let other task run.

→ sync and async code can be single threaded or multiple threaded

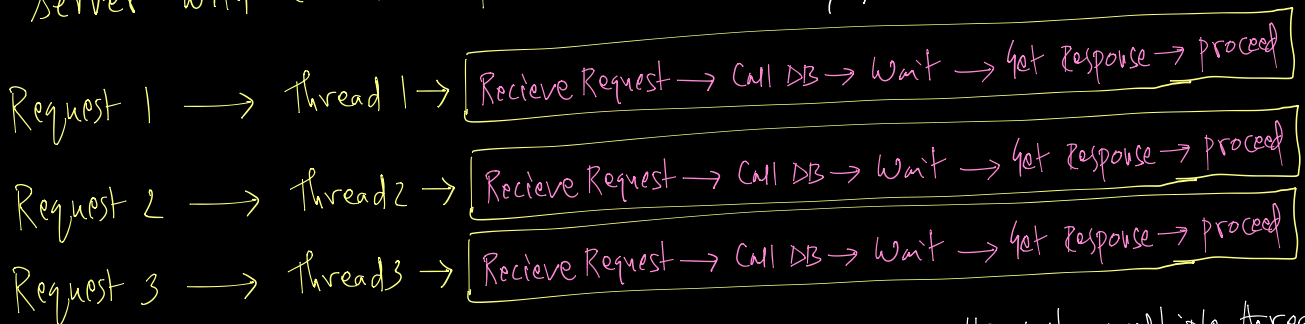
↓
Code structure

↓
How its executed

Example 1 Multithreading but Synchronous

→ Server with a thread pool

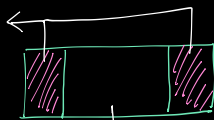
→ Synchronous process



Handling multiple request concurrently through multiple threads but code is synchronous

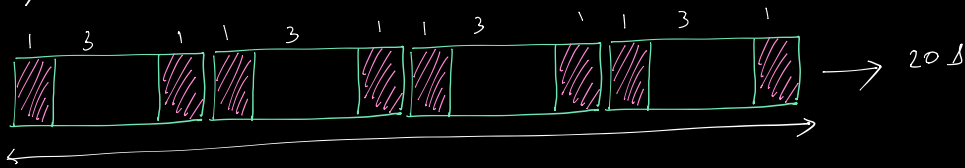
Example 2 → Single thread but asynchronous → Asyncio in Python

CPU Bound

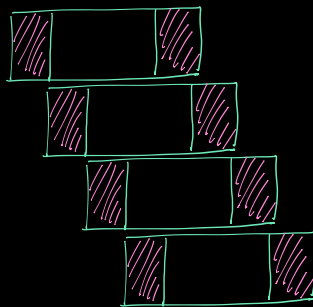


→ Typical Task.

Sequential
(Synchronous)



Concurrent
(Multithreads or Coroutines)



"Interleaving"

→ 5-6s

Concurrent Work

(Multiple task whose progress can overlap)

Preemptive

(Multithreading)

External Scheduler (like OS)
makes the switch between tasks

OS decides when to switch

Cooperative

Asynchronous Programming

Task gives control back voluntarily

(Python → Asyncio module) → COROUTINES

Single thread
event loop

← async / await →

Code decides when to switch

Multithreading

(Preemptive)

Thread A
Thread B



OS makes switch.

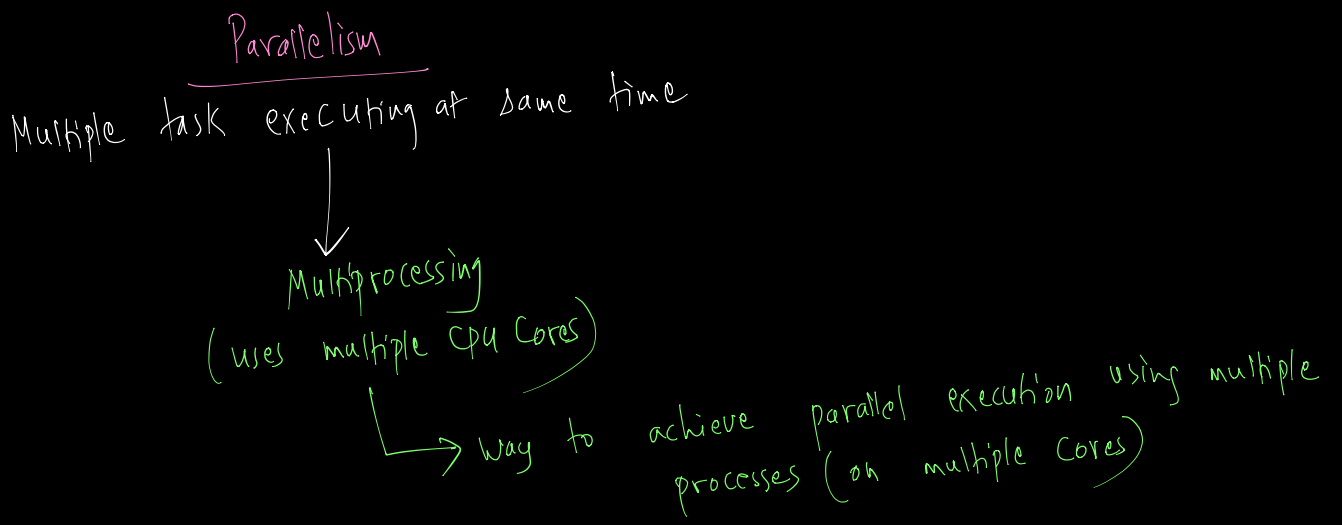
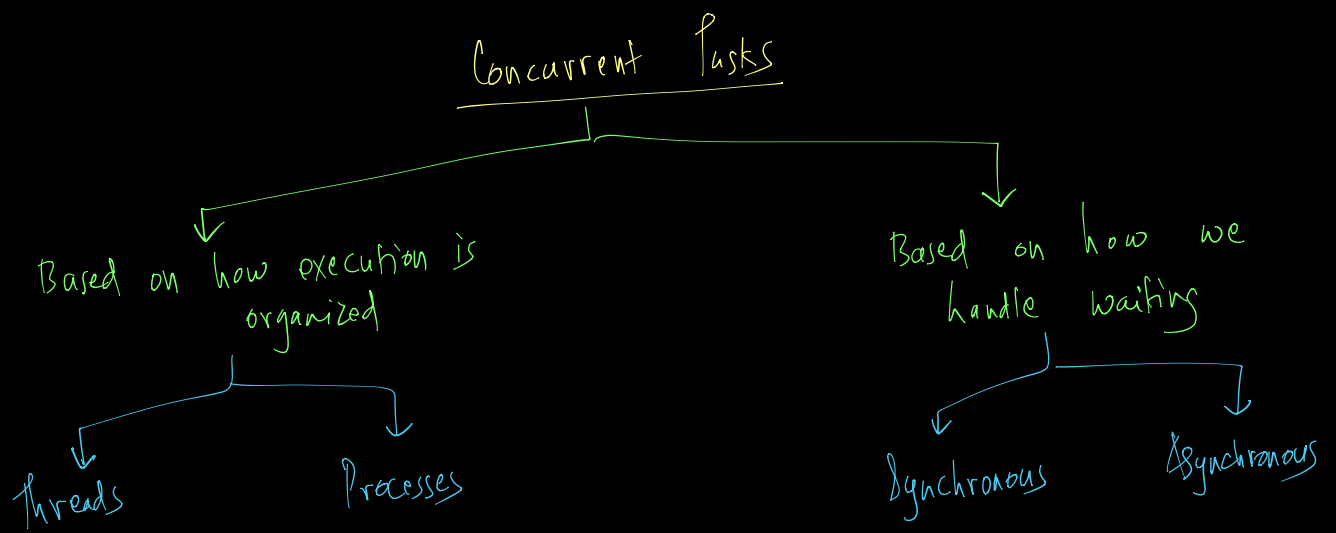
Async Programming
(here asyncio)

(Cooperative)

Task A
Task B



"await" on Task A by programmer
Event loop runs Task B



Parallelism can also be achieved through multithreading

↳ But GIL restricts it in standard CPython.

✓ Handling Concurrency → Multithreading [Preemptive] → Threads handled by OS
→ Asynchronous Programming
Asyncio [Cooperative] → Single Thread Concurrency

✓ Handling Parallel Execution → Multiprocessing

✓ Multithreading can also provide parallelism → In CPython not in Jython or IronPython
↳ But not in Python [GIL] (normally)
↓
Global Interpreter Lock
←
Only one thread can execute python bytecode at a time.

Programming paradigm where one task can voluntarily give control "while its waiting" to another task to make progress → Cooperative Concurrency

Multi threading is about workers & Asynchronous is about tasks

Common Confusions!

- ✓ ① Multithreading v/s Multiprocessing
- ✓ ② Multithreading v/s Asynchronous Programming
- ✓ ③ Multithreading v/s Asyncio.
- ✓ ④

Async + Single Threaded	] All possible but some are more common
Async + Multi Threaded	
Sync + Single Threaded	
Sync + Multithreaded.	